MusicRoom.WebAPI | v1

Generated from docs/api/openapi.json — version 1.0.0

Contents

API reference **1**

Routes 1

Schemas 26

API reference

Generated from `docs/api/openapi.json`, which is generated from the code and guarded against drift by `make agent-check`. **Nothing here is written by hand**, so nothing here can be out of date with the API while claiming otherwise.

It says what is on the wire. What a route *means* — which rule decided its status codes, why a refusal says what it says — is `docs/api/ENDPOINTS.md`, and that document is the contract.

Routes

`GET /oauth2/authorize`

Authorization endpoint — authorization code flow with PKCE

Takes `response_type=code`, `client_id`, `redirect_uri`, `scope`, `state`, `nonce`, `code_challenge` and `code_challenge_method=S256`, and answers 302 to the registered redirect URI with a single-use code that expires in 60 seconds. It requires a bearer access token: this provider has no login page, so the account signs in through the credential endpoints first — SPEC-044, the owner's answer A. An unknown client or an unregistered redirect URI answers 400 without redirecting, because sending an error to a URI nobody registered is the open redirect the registry exists to prevent.

Status	Meaning	Body
`302`	Found	no body
`400`	Bad Request	no body
`401`	Unauthorized	no body

`POST /oauth2/token`

Token endpoint — code exchange and refresh

Form-encoded, as RFC 6749 requires. With `grant_type=authorization_code` it takes `code`, `code_verifier`, `client_id` and `redirect_uri`, spends the code once, and answers an access token, an ID token and a refresh token. With `grant_type=refresh_token` it takes `refresh_token` and rotates the session, which revokes the whole family if a spent token is presented. There is no `grant_type=password`: OAuth 2.1 removes it. Errors are RFC 6749 §5.2 objects, not ProblemDetails.

Status	Meaning	Body
200	OK	`TokenGrantResponse`
400	Bad Request	`OAuthErrorResponse`
401	Unauthorized	`OAuthErrorResponse`
429	Too Many Requests	no body

`GET /oauth2/userinfo`

UserInfo endpoint

The claims the session's grant allows — OpenID Connect Core 1.0 §5.3. `sub` always; `email` and `email_verified` when the `email` scope was granted; `name` when `profile` was. A session opened by the credential endpoints granted no scope and receives all of them, because the caller is the account's own client. The subject comes from the validated token and never from a parameter.

Status	Meaning	Body
200	OK	`UserInfoResponse`
401	Unauthorized	no body

`POST /oauth2/revoke`

Revocation endpoint

Form-encoded `token`, with an optional `token_type_hint` — RFC 7009. Revoking a refresh token ends its whole session: every token descended from the same sign-in stops rotating. An unknown or already-revoked token answers 200 as well, so the endpoint is not an oracle for which tokens exist. `token_type_hint=access_token` answers 400 `unsupported_token_type`, because a short-lived signed access token is not revocable here and a 200 would say otherwise.

Status	Meaning	Body
200	OK	no body
400	Bad Request	`OAuthErrorResponse`
429	Too Many Requests	no body

`GET /.well-known/openid-configuration`

OpenID Connect discovery document

What this provider offers. Only endpoints that answer are listed: a client reading this document may assume every URL in it exists.

Status	Meaning	Body
200	OK	`DiscoveryDocument`

`GET /.well-known/jwks.json`

Public signing keys

The public half of every key a token from this API may carry: the current one, and the outgoing one during a rotation overlap. Private material never appears here.

Status	Meaning	Body
200	OK	no body

`GET /health`

Liveness

Returns 200 while the process is serving. Touches no dependency.

Status	Meaning	Body
200	OK	`object`

`GET /health/ready`

Readiness

Returns 200 when every dependency is reachable, 503 when one is not.

Status	Meaning	Body
200	OK	`object`
503	Service Unavailable	`object`

`POST /api/v1/me/password`

Change the caller's password

Takes the current password as proof and replaces it. Every other session the account holds is revoked; the caller's own stays alive, because it is the session that just proved it holds the new credential. A wrong current password is 422, not 401: the token is fine.

Request body: `ChangePasswordCommand`

Status	Meaning	Body
204	No Content	no body
401	Unauthorized	no body
422	Unprocessable Entity	no body
429	Too Many Requests	no body

`GET /api/v1/me/control-grants`

List the devices the caller has been given control of

The delegation seen from the other side: what somebody else handed me, not what I handed out.

Status	Meaning	Body
200	OK	array of `ControlGrantSummary`
401	Unauthorized	no body

`POST /api/v1/auth/registrations`

Register an account

Accepts the registration and answers 202 whether or not the address is already registered. A 409 here would tell an anonymous caller who holds an account; the mailbox tells the real user what happened.

Request body: `RegistrationRequest`

Status	Meaning	Body
202	Accepted	no body
400	Bad Request	no body
422	Unprocessable Entity	no body
429	Too Many Requests	no body

`POST /api/v1/auth/email-verifications`

Verify an email address

Spends a verification credential exactly once and marks the address verified. Send either `token`, from the link, or `email` with the six-character `code` from the message. A wrong code costs one of five attempts against that message, after which it is spent whatever is typed. The description says nothing about what separates 400 from 410 for a given credential, because that is the caller's business only when they hold it.

Request body: `EmailVerificationRequest`

Status	Meaning	Body
204	No Content	no body
400	Bad Request	no body
410	Gone	no body
429	Too Many Requests	no body

`POST /api/v1/auth/verification-requests`

Ask for another email verification link

Answers 202 for any address that is a valid address. Whether a mail follows is not stated and cannot be inferred from the response: a distinct answer here would tell an anonymous caller which addresses hold a registered or confirmed account.

Request body: `VerificationRequest`

Status	Meaning	Body
202	Accepted	no body
400	Bad Request	no body
422	Unprocessable Entity	no body
429	Too Many Requests	no body

`GET /api/v1/auth/sessions`

List your own sessions

Lists the caller's live sessions, newest first, each with the platform, device and app version that started it so a session can be recognised. No token and no digest is returned: the list exists to be read, not to be replayed.

Status	Meaning	Body
200	OK	array of `SessionResponse`
401	Unauthorized	no body

`POST /api/v1/auth/sessions`

Sign in

Exchanges an address and a password for a token pair. Every failure — unknown address, wrong password, unverified account — answers the same bodiless 401, and the unknown-address path spends the same hashing work as the others, so the response cannot be timed either.

Request body: `SessionRequest`

Status	Meaning	Body
200	OK	`TokenPairResponse`
400	Bad Request	no body
401	Unauthorized	no body
429	Too Many Requests	no body

`POST /api/v1/auth/tokens`

Rotate a session

Spends a refresh token exactly once and returns a rotated pair. Presenting an already-rotated token revokes the whole family and answers 401: that is the session-theft detector, and it answers exactly as an unknown or expired token does.

Request body: `TokenRotationRequest`

Status	Meaning	Body
200	OK	`TokenPairResponse`
400	Bad Request	no body
401	Unauthorized	no body
429	Too Many Requests	no body

`POST /api/v1/auth/password-reset-requests`

Ask for a password reset link

Answers 202 for any address that is a valid address. Whether a mail follows is not stated and cannot be inferred from the response: a distinct answer here would tell an anonymous caller which addresses hold an account.

Request body: `PasswordResetRequest`

Status	Meaning	Body
202	Accepted	no body
400	Bad Request	no body
422	Unprocessable Entity	no body
429	Too Many Requests	no body

`POST /api/v1/auth/password-resets`

Replace a forgotten password

Takes either the token from the reset link or the address and the six-character code from the same mail, spends it exactly once, replaces the password and revokes every refresh token the account holds, so a password change ends every session opened with the old one. Five wrong codes spend the row. An access token issued moments earlier stays valid until it expires; that window is the 15 minutes the session design fixes. A 422 describes the new password only, never the credential.

Request body: `PasswordResetSubmission`

Status	Meaning	Body
`204`	No Content	no body
`400`	Bad Request	no body
`410`	Gone	no body
`422`	Unprocessable Entity	no body
`429`	Too Many Requests	no body

`DELETE /api/v1/auth/sessions/current`

Sign out

Revokes the refresh-token family this access token belongs to, named by its sid claim, so no token from this session can be rotated again. The access token already issued stays valid until it expires — at most 15 minutes and 30 seconds — which is the accepted window in the threat register rather than an oversight.

Status	Meaning	Body
`204`	No Content	no body
`400`	Bad Request	no body
`401`	Unauthorized	no body

`DELETE /api/v1/auth/sessions/{sessionId}`

End one of your sessions

Revokes one named session of the calling account. An unknown id, one already ended and one belonging to another account all answer 404, because a distinct answer would confirm that another account's session exists.

Parameters:

- `sessionId` in path, required — `string` (uuid)

Status	Meaning	Body
`204`	No Content	no body
`400`	Bad Request	no body
`401`	Unauthorized	no body
`404`	Not Found	no body

`POST /api/v1/auth/sessions/social`

Sign in with a provider

Exchanges an authorization code for a token pair. The code is exchanged by this API with the client secret, never by the client — ADR-0008. An account is created the first time an identity signs in, and it has no password until the owner sets one through the reset flow.

Request body: `SocialSessionRequest`

Status	Meaning	Body
200	OK	`TokenPairResponse`
400	Bad Request	no body
401	Unauthorized	no body
422	Unprocessable Entity	no body
429	Too Many Requests	no body

`GET /api/v1/auth/social-links`

List linked providers

The caller's own links, oldest first. No route reads anybody else's.

Status	Meaning	Body
200	OK	array of `SocialLinkResponse`
401	Unauthorized	no body

`POST /api/v1/auth/social-links`

Link a provider to this account

The same acquisition as signing in, deliberately: linking an identity the caller cannot prove they hold would let anyone attach their provider account to somebody else's.

Request body: `SocialLinkRequest`

Status	Meaning	Body
201	Created	no body
401	Unauthorized	no body
409	Conflict	no body
422	Unprocessable Entity	no body

`DELETE /api/v1/auth/social-links/{provider}`

Unlink a provider

Refused with 409 when it is the account's only way in: an account with no password and one link would become one nobody can reach, including its owner.

Parameters:

- `provider` in path, required — `string`

Status	Meaning	Body
204	No Content	no body
401	Unauthorized	no body
409	Conflict	no body

`GET /api/v1/me`

Read your own profile

Returns every profile field with the visibility tier that governs it, plus the caller's music preferences. The owner is the one reader who sees every tier; what another user sees is projected per field, which is a different endpoint.

Status	Meaning	Body
200	OK	`OwnProfileResponse`
401	Unauthorized	no body

`PATCH /api/v1/me/profile`

Update your own profile

Updates any subset of the profile. Each field carries its value and its visibility tier in the same object, so a field cannot exist without a tier — a value with no tier is 422, never a silent default to public. An absent field is left untouched; sending a null value with a tier clears it.

Request body: `UpdateProfileRequest`

Status	Meaning	Body
200	OK	no body
400	Bad Request	no body
401	Unauthorized	no body
422	Unprocessable Entity	no body

`PUT /api/v1/me/music-preferences`

Replace your music preferences

Replaces the whole preference set, genres and artists. Preferences are private to their owner and carry no visibility tier of their own. Both lists are bounded: an unbounded array behind an authenticated endpoint is unrestricted resource consumption.

Request body: `MusicPreferencesRequest`

Status	Meaning	Body
200	OK	no body
400	Bad Request	no body
401	Unauthorized	no body
422	Unprocessable Entity	no body

`GET /api/v1/me/friends`

List your friends

Lists the caller's friends, newest first, paged. A friendship is symmetric: both parties see each other here, and neither can see it without the other.

Parameters:

- `page` in query, optional — `['integer', 'string']` (int32)
- `pageSize` in query, optional — `['integer', 'string']` (int32)

Status	Meaning	Body
200	OK	array of `FriendResponse`
401	Unauthorized	no body

`DELETE /api/v1/me/friends/{userId}`

Remove a friend

Ends the friendship for both sides at once, and clears the request rows between the pair so they may befriend again. Answers 404 when there is no friendship to end.

Parameters:

- `userId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body

`GET /api/v1/me/friend-requests`

List your pending friend requests

Lists pending requests in one direction: incoming are the ones addressed to the caller, outgoing the ones the caller sent. The direction is required, because the two screens that show these lists show one each.

Parameters:

- `direction` in query, optional — `string`
- `page` in query, optional — `['integer', 'string']` (int32)
- `pageSize` in query, optional — `['integer', 'string']` (int32)

Status	Meaning	Body
200	OK	array of `FriendRequestResponse`
401	Unauthorized	no body

`POST /api/v1/me/friend-requests`

Ask somebody to be friends

Creates a pending request. Answers 409 when the pair are already friends, when a request is already pending in either direction, or when the caller asks themselves; 404 when no such account exists. A crossing request is not auto-accepted: nobody becomes a friend without accepting.

Request body: `FriendRequestCreation`

Status	Meaning	Body
201	Created	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body
409	Conflict	no body

Status	Meaning	Body
422	Unprocessable Entity	no body

`PATCH /api/v1/me/friend-requests/{requestId}`

Accept or decline a friend request

Only the recipient may answer. Accepting creates the friendship in the same transaction that marks the request accepted, so twenty simultaneous accepts produce one friendship. An unknown request, one already answered and one addressed to somebody else all answer 404.

Parameters:

- `requestId` in path, required — `string` (uuid)

Request body: `FriendRequestDecision`

Status	Meaning	Body
200	OK	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body
422	Unprocessable Entity	no body

`DELETE /api/v1/me/friend-requests/{requestId}`

Withdraw a friend request you sent

Only the sender may withdraw; a recipient declines instead, which is a different verb and a different row state. The row is deleted rather than marked, so the pair may try again.

Parameters:

- `requestId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body

`GET /api/v1/users`

Search accounts by display name

Matches a case-insensitive prefix of the display name, paged with a capped size. The display name is the only field searched: matching a profile field would let a stranger confirm a friends-only value by probing for it. A blank query returns nothing rather than everybody.

Parameters:

- `q` in query, optional — `string`
- `page` in query, optional — `['integer', 'string']` (int32)
- `pageSize` in query, optional — `['integer', 'string']` (int32)

Status	Meaning	Body
200	OK	array of `UserSearchResult`
401	Unauthorized	no body

`GET /api/v1/users/{userId}`

Read one account's profile

Returns the profile projected per field against the caller's relationship to its owner: self, friend or stranger. A field the caller may not see is absent from the body and is never read from the database, so a caller cannot tell a hidden field from an empty one. An account that does not exist answers the same sparse shape as one whose owner hid everything, so asking does not confirm existence.

Parameters:

- `userId` in path, required — `string` (uuid)

Status	Meaning	Body
200	OK	`VisibleProfile`
401	Unauthorized	no body

`GET /api/v1/playlists`

List playlists

One scope at a time: mine, shared with me, or public. Paged with a capped size. A private playlist the caller is not named on never appears in any scope.

Parameters:

- `scope` in query, optional — `string`
- `page` in query, optional — `['integer', 'string']` (int32)
- `pageSize` in query, optional — `['integer', 'string']` (int32)

Status	Meaning	Body
200	OK	array of `PlaylistSummary`
401	Unauthorized	no body

`POST /api/v1/playlists`

Create a playlist

The caller owns it. A playlist may be Everyone or InvitedOnly; LocationAndTimeRestricted is 422, because that licence belongs to an event and nothing checks a geofence on a playlist.

Request body: `PlaylistCreation`

Status	Meaning	Body
201	Created	no body
400	Bad Request	no body
401	Unauthorized	no body
422	Unprocessable Entity	no body

`GET /api/v1/playlists/{playlistId}`

Read one playlist

Returns the playlist with an ETag the client sends back as If-Match when it edits. A playlist the caller may not see answers 404, never 403, because 403 would confirm it exists.

Parameters:

- `playlistId` in path, required — `string` (uuid)

Status	Meaning	Body
200	OK	`PlaylistResponse`
401	Unauthorized	no body
404	Not Found	no body

`PATCH /api/v1/playlists/{playlistId}`

Change a playlist's metadata

Requires If-Match carrying the ETag the client last read. A stale or absent token answers 412 and changes nothing: two clients editing at once is the failure this guard exists for.

Parameters:

- `playlistId` in path, required — `string` (uuid)

Request body: `PlaylistMetadata`

Status	Meaning	Body
200	OK	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body
412	Precondition Failed	no body
422	Unprocessable Entity	no body

`DELETE /api/v1/playlists/{playlistId}`

Delete a playlist

A soft delete, owner only: the row is stamped deleted and disappears from every read. Its audit trail keeps pointing at something that exists.

Parameters:

- `playlistId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body

`PUT /api/v1/playlists/{playlistId}/licence`

Set a playlist's visibility and licence

Owner only: a collaborator who may edit the contents does not decide who may see them.

Parameters:

- `playlistId` in path, required — `string` (uuid)

Request body: `PlaylistLicenceRequest`

Status	Meaning	Body
200	OK	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body
422	Unprocessable Entity	no body

`GET /api/v1/playlists/{playlistId}/collaborators`

List who is named on a playlist

Anybody who may see the playlist may see who is named on it. A caller who may not see the playlist answers 404, not an empty list, because an empty list would confirm it exists.

Parameters:

- `playlistId` in path, required — `string` (uuid)

Status	Meaning	Body
200	OK	array of `CollaboratorSummary`
401	Unauthorized	no body
404	Not Found	no body

`POST /api/v1/playlists/{playlistId}/collaborators`

Name somebody on a playlist

Owner only. 409 when they are already named, and when the owner names themselves — they already hold every right a collaborator grant could give.

Parameters:

- `playlistId` in path, required — `string` (uuid)

Request body: `CollaboratorCreation`

Status	Meaning	Body
201	Created	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body
409	Conflict	no body

`DELETE /api/v1/playlists/{playlistId}/collaborators/{userId}`

Remove somebody from a playlist

Owner only. 404 when there is no such grant on a playlist this caller owns.

Parameters:

- `playlistId` in path, required — `string` (uuid)
- `userId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body

`GET /api/v1/playlists/{playlistId}/tracks`

List a playlist's tracks in order

Anybody who may see the playlist may see its tracks. The response carries the ETag the client sends back as If-Match when it adds, moves or removes one. A caller who may not see the playlist answers 404, not an empty list, because an empty list would confirm it exists.

Parameters:

- `playlistId` in path, required — `string` (uuid)
- `page` in query, optional — `['integer', 'string']` (int32)
- `pageSize` in query, optional — `['integer', 'string']` (int32)

Status	Meaning	Body
200	OK	array of `PlaylistTrackSummary`
401	Unauthorized	no body
404	Not Found	no body

`POST /api/v1/playlists/{playlistId}/tracks`

Add a track to a playlist

Requires If-Match carrying the ETag of the track list the client last read. Appends by default; atHead or afterTrackId names a position. A playlist already holding its maximum number of tracks answers 409. The same video may appear twice: a playlist that plays a track twice is a playlist, not an error.

Parameters:

- `playlistId` in path, required — `string` (uuid)

Request body: `TrackAddition`

Status	Meaning	Body
201	Created	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body
409	Conflict	no body
412	Precondition Failed	no body

Status	Meaning	Body
422	Unprocessable Entity	no body

`PATCH /api/v1/playlists/{playlistId}/tracks/{playlistTrackId}`

Move a track within a playlist

Exactly one of afterTrackId or beforeTrackId. Requires If-Match: when two clients reorder the same list at once exactly one is applied and the other is told 412 and reloads, which is the whole point of versioning a collection.

Parameters:

- `playlistId` in path, required — `string` (uuid)
- `playlistTrackId` in path, required — `string` (uuid)

Request body: `TrackDestination`

Status	Meaning	Body
200	OK	no body
400	Bad Request	no body
401	Unauthorized	no body
404	Not Found	no body
412	Precondition Failed	no body
422	Unprocessable Entity	no body

`DELETE /api/v1/playlists/{playlistId}/tracks/{playlistTrackId}`

Remove a track from a playlist

Requires If-Match. Removing leaves the remaining order untouched: positions are sparse, so the hole costs nothing and no other row is rewritten.

Parameters:

- `playlistId` in path, required — `string` (uuid)
- `playlistTrackId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
401	Unauthorized	no body
404	Not Found	no body
412	Precondition Failed	no body

`GET /api/v1/events`

List events

One scope at a time: mine, the ones I was invited to, or public. Paged with a capped size. A private event the caller was not invited to appears in none of them.

Parameters:

- `scope` in query, optional — `string`
- `page` in query, optional — `['integer', 'string']` (int32)
- `pageSize` in query, optional — `['integer', 'string']` (int32)

Status	Meaning	Body
200	OK	array of EventSummary
401	Unauthorized	no body

`POST /api/v1/events`

Create an event

The caller owns it. The window is two UTC instants and the end must be after the start. A LocationAndTimeRestricted licence must name the presence proof it requires, and a geofence proof must carry a centre and a radius, or the answer is 422.

Request body: `EventCreation`

Status	Meaning	Body
201	Created	no body
400	Bad Request	no body
401	Unauthorized	no body
422	Unprocessable Entity	no body

`GET /api/v1/events/{eventId}`

Read one event

Returns the event with an ETag the client sends back as If-Match when it edits. An event the caller may not see answers 404, never 403, because 403 would confirm it exists.

Parameters:

- `eventId` in path, required — `string` (uuid)

Status	Meaning	Body
200	OK	EventResponse
401	Unauthorized	no body
404	Not Found	no body

`PATCH /api/v1/events/{eventId}`

Change an event's metadata

Owner only, and requires If-Match carrying the ETag the client last read. A stale or absent token answers 412 and changes nothing.

Parameters:

- `eventId` in path, required — `string` (uuid)

Request body: `EventMetadataRequest`

Status	Meaning	Body
200	OK	no body
401	Unauthorized	no body
404	Not Found	no body
412	Precondition Failed	no body
422	Unprocessable Entity	no body

Status	Meaning	Body

`DELETE /api/v1/events/{eventId}`

Delete an event

A soft delete, owner only: the row is stamped deleted and disappears from every read, so its audit trail keeps pointing at something that exists.

Parameters:

- `eventId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
401	Unauthorized	no body
404	Not Found	no body

`PUT /api/v1/events/{eventId}/licence`

Set an event's visibility, licence and presence policy

Owner only. The same rules creation applies: an event cannot become a LocationAndTimeRestricted event with nothing configured to check.

Parameters:

- `eventId` in path, required — `string` (uuid)

Request body: `EventLicenceRequest`

Status	Meaning	Body
200	OK	no body
401	Unauthorized	no body
404	Not Found	no body
422	Unprocessable Entity	no body

`GET /api/v1/events/{eventId}/invitations`

List who was invited

Owner only. Who else was invited to a private event is not a participant's business, and an empty list for a non-owner would still confirm the event exists — so the answer is 404.

Parameters:

- `eventId` in path, required — `string` (uuid)

Status	Meaning	Body
200	OK	array of `InvitationSummary`
401	Unauthorized	no body
404	Not Found	no body

`POST /api/v1/events/{eventId}/invitations`

Invite somebody to an event

Owner only. A duplicate is 409, decided by the unique index rather than by a read-then-write check. Inviting the owner is also 409: they already hold the strongest grant.

Parameters:

- `eventId` in path, required — `string` (uuid)

Request body: `InvitationRequest`

Status	Meaning	Body
201	Created	no body
401	Unauthorized	no body
404	Not Found	no body
409	Conflict	no body

`DELETE /api/v1/events/{eventId}/invitations/{userId}`

Withdraw an invitation

Owner only. The gate is read live on every request, so the invitee loses access on their next call rather than on their next sign-in.

Parameters:

- `eventId` in path, required — `string` (uuid)
- `userId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
401	Unauthorized	no body
404	Not Found	no body

`POST /api/v1/events/{eventId}/attendances`

Join an event's roster

Any authenticated caller on a public event, an invitee on a private one. This is a roster and never a precondition: voting does not require it.

Parameters:

- `eventId` in path, required — `string` (uuid)

Status	Meaning	Body
201	Created	no body
401	Unauthorized	no body
404	Not Found	no body
409	Conflict	no body

`DELETE /api/v1/events/{eventId}/attendances/current`

Leave an event's roster

`current` rather than a user id: the only attendance a caller may remove is their own.

Parameters:

- `eventId` in path, required — `string` (uuid)

Status	Meaning	Body
`204`	No Content	no body
`401`	Unauthorized	no body
`404`	Not Found	no body

`GET /api/v1/events/{eventId}/queue`

Read the vote-ordered queue

Anybody who may see the event may see its queue. Ordered by vote count and then by when each track was suggested, so the order is total and stable. Each row says whether the caller voted, and the count is read from the vote rows rather than from a stored counter.

Parameters:

- `eventId` in path, required — `string` (uuid)
- `page` in query, optional — `['integer', 'string']` (int32)
- `pageSize` in query, optional — `['integer', 'string']` (int32)

Status	Meaning	Body
`200`	OK	array of `QueueEntry`
`401`	Unauthorized	no body
`404`	Not Found	no body

`POST /api/v1/events/{eventId}/tracks`

Suggest a track for an event

Behind the licence gate: who may suggest is who may vote. The same video twice is 409, because two rows would split its votes between two queue entries.

Parameters:

- `eventId` in path, required — `string` (uuid)

Request body: `SuggestionRequest`

Status	Meaning	Body
`201`	Created	no body
`401`	Unauthorized	no body
`404`	Not Found	no body
`409`	Conflict	no body
`422`	Unprocessable Entity	no body

`DELETE /api/v1/events/{eventId}/tracks/{suggestionId}`

Remove a suggestion

The suggester's or the event owner's, and nobody else's. Its votes go with it: a vote is a vote for that suggestion, and orphaned rows would leave a count that counts nothing.

Parameters:

- `eventId` in path, required — `string` (uuid)
- `suggestionId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
401	Unauthorized	no body
404	Not Found	no body

`PUT /api/v1/events/{eventId}/tracks/{suggestionId}/vote`

Vote for a suggestion

Idempotent: 201 the first time and 200 after, including when a concurrent request wrote the vote first — the unique index is the arbiter. On a LocationAndTimeRestricted event the body carries presence, and a missing or failed proof is 422. The coordinate is compared and discarded; it is never stored and never logged.

Parameters:

- `eventId` in path, required — `string` (uuid)
- `suggestionId` in path, required — `string` (uuid)

Request body: `VoteRequest` or null

Status	Meaning	Body
200	OK	no body
201	Created	no body
401	Unauthorized	no body
404	Not Found	no body
422	Unprocessable Entity	no body
429	Too Many Requests	no body

`DELETE /api/v1/events/{eventId}/tracks/{suggestionId}/vote`

Take back a vote

The caller's own vote and no other. No presence proof is asked for: leaving the venue is not a reason to be stuck having voted, and a withdrawal lowers a count rather than raising one.

Parameters:

- `eventId` in path, required — `string` (uuid)
- `suggestionId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
401	Unauthorized	no body
404	Not Found	no body

`POST /api/v1/events/{eventId}/event-codes`

Issue a short-lived venue code

Owner only, and rate limited per account. The code is returned once and stored only as a SHA-256 digest; it is single-use per account, so one code works for a whole room exactly once each.

Parameters:

- `eventId` in path, required — `string` (uuid)

Request body: `EventCodeRequest` or null

Status	Meaning	Body
201	Created	no body
401	Unauthorized	no body
404	Not Found	no body
422	Unprocessable Entity	no body
429	Too Many Requests	no body

`GET /api/v1/events/{eventId}/playback-session`

Read what an event is playing

Anybody who may see the event may see what it is playing, which is the rule GET /playback-sessions/{sessionId} already applies — this route is for the guest who has the event and not the session id. The answer carries nowPlayingStartedAtUtc, so a guest arriving mid-track starts from the right second rather than from the beginning, and serverNowUtc, so they can correct for their own clock. An event with nothing playing, and an event the caller may not see, both answer 404. When two devices are playing one event, the newest live session answers.

Parameters:

- `eventId` in path, required — `string` (uuid)

Status	Meaning	Body
200	OK	`PlaybackSessionDetail`
401	Unauthorized	no body
404	Not Found	no body

`GET /api/v1/devices`

List the caller's own devices

Only the caller's. The route takes no account id, because a route that accepted one would be a route somebody would eventually pass somebody else's.

Status	Meaning	Body
200	OK	array of `DeviceSummary`
401	Unauthorized	no body

`POST /api/v1/devices`

Register the calling device

Takes no body: the platform, the model and the app version come from the mandatory client headers. Registering the same device twice is 409, where the same device means the same owner, platform and model — a phone that updates its app is the same phone.

Status	Meaning	Body
`201`	Created	no body
`400`	Bad Request	no body
`401`	Unauthorized	no body
`409`	Conflict	no body

`DELETE /api/v1/devices/{deviceId}`

Remove one of the caller's devices

Owner only. Another account's device answers 404, never 403, because 403 would confirm it exists.

Parameters:

- `deviceId` in path, required — `string` (uuid)

Status	Meaning	Body
`204`	No Content	no body
`401`	Unauthorized	no body
`404`	Not Found	no body

`GET /api/v1/devices/{deviceId}/control`

Read who controls a device

The owner or the current controller may ask; anybody else gets 404. With no grant the controller is the owner. The response carries the ETag a claim sends back as If-Match.

Parameters:

- `deviceId` in path, required — `string` (uuid)

Status	Meaning	Body
`200`	OK	`DeviceControl`
`401`	Unauthorized	no body
`404`	Not Found	no body

`PUT /api/v1/devices/{deviceId}/control`

Hand playback control to a friend

Owner only, and requires If-Match. The grantee must be a friend of the owner, or the answer is 422. Two simultaneous claims: one 200, one 412 — the loser loses visibly, because two clients each believing they hold one device is the bug.

Parameters:

- `deviceId` in path, required — `string` (uuid)

Request body: `ControlGrantRequest`

Status	Meaning	Body
200	OK	no body
401	Unauthorized	no body
404	Not Found	no body
412	Precondition Failed	no body
422	Unprocessable Entity	no body

`DELETE /api/v1/devices/{deviceId}/control`

Return a device to its owner's control

The owner's revoke and the controller's release are the same route and the same effect. A device already under owner control answers 404: there was no grant to release.

Parameters:

- `deviceId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
401	Unauthorized	no body
404	Not Found	no body

`GET /api/v1/devices/{deviceId}/playback-session`

Read what a device is playing

The device's owner or its current controller: the people who could change what it plays are the people entitled to know what it is playing. A device that is not playing answers 404.

Parameters:

- `deviceId` in path, required — `string` (uuid)

Status	Meaning	Body
200	OK	`PlaybackSessionDetail`
401	Unauthorized	no body
404	Not Found	no body

`GET /api/v1/tracks`

Search the catalog

The only route that can reach YouTube's expensive search call, so it is rate limited per account and its results are cached. A repeated query inside the cache window costs no quota at all.

Parameters:

- `q` in query, optional — `string`
- `page` in query, optional — `['integer', 'string']` (int32)
- `pageSize` in query, optional — `['integer', 'string']` (int32)

Status	Meaning	Body
200	OK	array of `TrackMetadata`

Status	Meaning	Body
401	Unauthorized	no body
422	Unprocessable Entity	no body
429	Too Many Requests	no body
503	Service Unavailable	no body

`GET /api/v1/tracks/{youtubeVideoId}`

Resolve one track

Prefers both cache levels, then one cheap metadata call. A video the catalog does not know is 404, and so is a malformed id — the contract lists no 422 here, and one answer for both keeps the route from saying which ids are well formed. A catalog that cannot answer at all is 503.

Parameters:

- `youtubeVideoId` in path, required — `string`

Status	Meaning	Body
200	OK	`TrackMetadata`
401	Unauthorized	no body
404	Not Found	no body
503	Service Unavailable	no body

`POST /api/v1/playback-sessions`

Start playing a source on a device

The device's owner starts playback, on a source they may see. The server picks what plays first. A device already playing answers 409: it holds at most one live session.

Request body: `PlaybackStartRequest`

Status	Meaning	Body
201	Created	no body
401	Unauthorized	no body
404	Not Found	no body
409	Conflict	no body

`GET /api/v1/playback-sessions/{sessionId}`

Read one playback session

Anybody who may see the source may see what it is playing, with the ETag a command sends back as If-Match. A caller who may not see the source answers 404.

Parameters:

- `sessionId` in path, required — `string` (uuid)

Status	Meaning	Body
200	OK	`PlaybackSessionDetail`
401	Unauthorized	no body
404	Not Found	no body

`PATCH /api/v1/playback-sessions/{sessionId}`

Jump to a named item

The one route where the client chooses what plays, so it is guarded and bounded: the controller may jump, and only to an item of the source already playing.

Parameters:

- `sessionId` in path, required — `string` (uuid)

Request body: `PlaybackJumpRequest`

Status	Meaning	Body
200	OK	no body
401	Unauthorized	no body
404	Not Found	no body
412	Precondition Failed	no body

`DELETE /api/v1/playback-sessions/{sessionId}`

Stop playback

The device's owner stops it: a delegate steers what plays and does not get to switch the speaker off. The row is stamped ended rather than deleted, and the device may start again.

Parameters:

- `sessionId` in path, required — `string` (uuid)

Status	Meaning	Body
204	No Content	no body
401	Unauthorized	no body
404	Not Found	no body

`POST /api/v1/playback-sessions/{sessionId}/advances`

Report that the track ended

The controller's route. The server picks the next item — highest vote count for an event, next position for a playlist. `expectedNowPlayingId` makes a retry harmless: a stale expectation returns the current state rather than skipping a second track. A stale If-Match is 412. Nothing left to play ends the session and answers 409.

Parameters:

- `sessionId` in path, required — `string` (uuid)

Request body: `PlaybackAdvanceRequest` or null

Status	Meaning	Body
200	OK	`PlaybackSessionDetail`
401	Unauthorized	no body
404	Not Found	no body
409	Conflict	no body
412	Precondition Failed	no body

`PUT /api/v1/playback-sessions/{sessionId}/position`

Report where the playing device is

The controller's route, as the jump is: only the thing playing the music can honestly say where it is. The server stamps its own clock on the report and broadcasts PlaybackPositionChanged, so a guest can seek to the same second. A report naming a track that is no longer playing is discarded — it was in flight while the session advanced, and acting on it would drag every listener backwards. An ended session, and a caller who may not command this one, both answer 404. The session's own ETag is untouched: a report is not a move, and a controller's If-Match keeps working. One report per second per session; above that is 429.

Parameters:

- `sessionId` in path, required — `string` (uuid)

Request body: `PlaybackPositionRequest`

Status	Meaning	Body
`204`	No Content	no body
`400`	Bad Request	no body
`401`	Unauthorized	no body
`404`	Not Found	no body
`429`	Too Many Requests	no body

Schemas

`ChangePasswordCommand`

Replaces the caller's password — `POST /api/v1/me/password`, SPEC-028.

Property	Type	Required
`currentPassword`	`string`	yes
`newPassword`	`string`	yes
`auditAction`	`['null', 'string']`	no
`auditTargetType`	`['null', 'string']`	no

`CollaboratorCreation`

The collaborator payload — `docs/api/ENDPOINTS.md` §5.

Property	Type	Required
`userId`	`string` (uuid)	yes
`canEdit`	`boolean`	yes

`CollaboratorSummary`

One person named on a playlist.

Property	Type	Required
`userId`	`string` (uuid)	yes
`displayName`	`string`	yes
`canEdit`	`boolean`	yes

`ControlGrantRequest`

The control grant payload — `docs/api/ENDPOINTS.md` §6.

Property	Type	Required
`granteeUserId`	`string` (uuid)	yes

`ControlGrantSummary`

One device somebody else has given this caller control of.

Property	Type	Required
`deviceId`	`string` (uuid)	yes
`ownerId`	`string` (uuid)	yes
`ownerDisplayName`	`string`	yes
`platform`	`string`	yes
`model`	`string`	yes

`CoordinatesRequest`

A claimed position. Compared and discarded.

Property	Type	Required
`latitude`	`['number', 'string']` (double)	yes
`longitude`	`['number', 'string']` (double)	yes

`DeviceControl`

Who controls a device, and the version a claim must send back.

Property	Type	Required
`deviceId`	`string` (uuid)	yes
`ownerId`	`string` (uuid)	yes
`controllerUserId`	`string` (uuid)	yes
`isDelegated`	`boolean`	yes
`version`	`['integer', 'string']` (uint32)	yes

`DeviceSummary`

One device, as its owner reads it.

Property	Type	Required
`deviceId`	`string` (uuid)	yes
`platform`	`string`	yes
`model`	`string`	yes
`appVersion`	`string`	yes
`controllerUserId`	`['null', 'string']` (uuid)	yes

`DiscoveryDocument`

The discovery document — OpenID Connect Discovery 1.0 §3, with only the members this provider can honour.

Property	Type	Required
`issuer`	`string`	yes
`jwks_uri`	`string`	yes
`authorization_endpoint`	`string`	yes
`token_endpoint`	`string`	yes
`userinfo_endpoint`	`string`	yes
`revocation_endpoint`	`string`	yes
`scopes_supported`	array of `string`	yes
`id_token_signing_alg_values_supported`	array of `string`	yes
`subject_types_supported`	array of `string`	yes
`response_types_supported`	array of `string`	yes
`grant_types_supported`	array of `string`	yes
`code_challenge_methods_supported`	array of `string`	yes

`EmailVerificationRequest`

The email verification payload — `docs/api/ENDPOINTS.md` §1. One credential or the other, SPEC-052.

Property	Type	Required
`token`	`['null', 'string']`	yes
`email`	`['null', 'string']`	no
`code`	`['null', 'string']`	no

`EventCodeRequest`

The code-issuing payload.

Property	Type	Required
`ttlSeconds`	`['null', 'integer', 'string']` (int32)	yes

`EventCreation`

The event creation payload — `docs/api/ENDPOINTS.md` §4.

Property	Type	Required
`name`	`string`	yes
`startsAtUtc`	`string` (date-time)	yes
`endsAtUtc`	`string` (date-time)	yes
`visibility`	`Visibility` or null	yes
`licence`	`LicenceMode` or null	yes
`presenceProof`	`PresenceProof` or null	yes
`geofence`	`GeofenceRequest` or null	yes

`EventLicenceRequest`

The licence payload.

Property	Type	Required
`visibility`	`Visibility`	yes
`licence`	`LicenceMode`	yes
`presenceProof`	`PresenceProof` or null	yes
`geofence`	`GeofenceRequest` or null	yes

`EventMetadataRequest`

The metadata payload for the guarded `PATCH`.

Property	Type	Required
`name`	`string`	yes
`startsAtUtc`	`string` (date-time)	yes
`endsAtUtc`	`string` (date-time)	yes

`EventResponse`

One event, as a client reads it.

Property	Type	Required
`eventId`	`string` (uuid)	yes
`name`	`string`	yes
`visibility`	`Visibility`	yes
`licence`	`LicenceMode`	yes
`startsAtUtc`	`string` (date-time)	yes
`endsAtUtc`	`string` (date-time)	yes
`ownerId`	`string` (uuid)	yes
`geofence`	`GeofenceRequest` or null	yes

`EventSummary`

One event in a list.

Property	Type	Required
`eventId`	`string` (uuid)	yes
`name`	`string`	yes
`visibility`	`Visibility`	yes
`startsAtUtc`	`string` (date-time)	yes
`endsAtUtc`	`string` (date-time)	yes
`ownerId`	`string` (uuid)	yes

`FriendRequestCreation`

The friend request payload — `docs/api/ENDPOINTS.md` §2.

Property	Type	Required
`targetUserId`	`string` (uuid)	yes

`FriendRequestDecision`

The response payload for accepting or declining.

Property	Type	Required
`status`	`string`	yes

`FriendRequestResponse`

One pending request, as a client reads it.

Property	Type	Required
`requestId`	`string` (uuid)	yes
`userId`	`string` (uuid)	yes
`displayName`	`string`	yes
`requestedAt`	`string` (date-time)	yes

`FriendResponse`

One friend, as a client reads them.

Property	Type	Required
`userId`	`string` (uuid)	yes
`displayName`	`string`	yes
`friendsSince`	`string` (date-time)	yes

`GeofenceRequest`

A venue and how close counts as present — ADR-0014.

Property	Type	Required
`latitude`	`['number', 'string']` (double)	yes
`longitude`	`['number', 'string']` (double)	yes
`radiusMetres`	`['integer', 'string']` (int32)	yes

`HttpValidationProblemDetails`

Property	Type	Required
`type`	`['null', 'string']`	no
`title`	`['null', 'string']`	no
`status`	`['null', 'integer', 'string']` (int32)	no
`detail`	`['null', 'string']`	no
`instance`	`['null', 'string']`	no
`errors`	`object`	no

Property	Type	Required
`codes`	`['null', 'object']`	no

`InvitationRequest`

The invitation payload.

Property	Type	Required
`userId`	`string` (uuid)	yes

`InvitationSummary`

One invitation.

Property	Type	Required
`userId`	`string` (uuid)	yes
`displayName`	`string`	yes

`LicenceMode`

Type: `integer`

`MusicPreferencesRequest`

The music preferences payload — `docs/api/ENDPOINTS.md` §2.

Property	Type	Required
`genres`	array of `string`	yes
`artists`	array of `string`	yes

`OAuthErrorResponse`

A refused grant — RFC 6749 §5.2.

Property	Type	Required
`error`	`string`	yes

`OwnProfileResponse`

A profile as its owner reads it.

Property	Type	Required
`displayName`	`string`	yes
`realName`	`ProfileFieldResponseOfstring`	yes
`bio`	`ProfileFieldResponseOfstring`	yes
`location`	`ProfileFieldResponseOfstring`	yes
`birthDate`	`ProfileFieldDateResponse`	yes
`genres`	array of `string`	yes

Property	Type	Required
`artists`	array of `string`	yes

`PasswordResetRequest`

The reset request payload — `docs/api/ENDPOINTS.md` §1.

Property	Type	Required
`email`	`string`	yes

`PasswordResetSubmission`

The reset payload — `docs/api/ENDPOINTS.md` §1.

Property	Type	Required
`newPassword`	`string`	yes
`token`	`['null', 'string']`	no
`email`	`['null', 'string']`	no
`code`	`['null', 'string']`	no

`PlaybackAdvanceRequest`

The advance payload.

Property	Type	Required
`expectedNowPlayingId`	`['null', 'string']` (uuid)	yes

`PlaybackJumpRequest`

The jump payload.

Property	Type	Required
`nowPlayingId`	`string` (uuid)	yes

`PlaybackPositionRequest`

One report from the playing device — `docs/api/ENDPOINTS.md` §7, SPEC-059.

Property	Type	Required
`nowPlayingId`	`string` (uuid)	yes
`positionMs`	`['integer', 'string']` (int32)	yes
`paused`	`boolean`	yes

`PlaybackSessionDetail`

One playback session, as a caller who may see its source reads it.

Property	Type	Required
sessionId	string (uuid)	yes
deviceId	string (uuid)	yes
sourceType	PlaybackSourceType	yes
sourceId	string (uuid)	yes
nowPlayingId	['null', 'string'] (uuid)	yes
startedAtUtc	string (date-time)	yes
endedAtUtc	['null', 'string'] (date-time)	yes
version	['integer', 'string'] (uint32)	yes
nowPlayingStartedAtUtc	['null', 'string'] (date-time)	yes
serverNowUtc	string (date-time)	yes
positionMs	['null', 'integer', 'string'] (int32)	no
positionAtUtc	['null', 'string'] (date-time)	no
paused	['null', 'boolean']	no

PlaybackSourceType

What a playback session is playing from — ADR-0017.

Type: integer

PlaybackStartRequest

The session-start payload — docs/api/ENDPOINTS.md §7.

Property	Type	Required
deviceId	string (uuid)	yes
sourceType	PlaybackSourceType	yes
sourceId	string (uuid)	yes

PlaylistCreation

The playlist creation payload — docs/api/ENDPOINTS.md §5.

Property	Type	Required
name	string	yes
visibility	Visibility or null	yes
licence	LicenceMode or null	yes

PlaylistLicenceRequest

The licence payload.

Property	Type	Required
visibility	Visibility	yes
licence	LicenceMode	yes

PlaylistMetadata

The metadata payload for the guarded PATCH.

Property	Type	Required
`name`	`string`	yes

`PlaylistResponse`

One playlist, as a client reads it.

Property	Type	Required
`playlistId`	`string` (uuid)	yes
`name`	`string`	yes
`visibility`	`Visibility`	yes
`licence`	`LicenceMode`	yes
`ownerId`	`string` (uuid)	yes
`mayEdit`	`boolean`	yes

`PlaylistSummary`

One playlist in a list.

Property	Type	Required
`playlistId`	`string` (uuid)	yes
`name`	`string`	yes
`visibility`	`Visibility`	yes
`ownerId`	`string` (uuid)	yes

`PlaylistTrackSummary`

One track in a playlist, as a client reads it — `GET /api/v1/playlists/{playlistId}/tracks`.

Property	Type	Required
`playlistTrackId`	`string` (uuid)	yes
`youtubeVideoId`	`string`	yes
`position`	`['integer', 'string']` (int64)	yes
`addedByUserId`	`string` (uuid)	yes

`PresenceProof`

Type: `integer`

`PresenceRequest`

The evidence a voter supplies — `presence` in `docs/api/ENDPOINTS.md` §4.

Property	Type	Required
`coordinates`	`CoordinatesRequest` or null	yes
`eventCode`	`['null', 'string']`	yes

`ProblemDetails`

Property	Type	Required
`type`	`['null', 'string']`	no
`title`	`['null', 'string']`	no
`status`	`['null', 'integer', 'string']` (int32)	no
`detail`	`['null', 'string']`	no
`instance`	`['null', 'string']`	no
`code`	`['null', 'string']`	no

`ProfileDateFieldRequest`

The date-of-birth field as a client sends it.

Property	Type	Required
`value`	`['null', 'string']` (date)	yes
`visibility`	`ProfileFieldVisibility` or null	yes

`ProfileFieldDateResponse`

The date-of-birth field and its tier.

Property	Type	Required
`value`	`['null', 'string']` (date)	yes
`visibility`	`ProfileFieldVisibility`	yes

`ProfileFieldRequestOfstring`

One field as a client sends it: a value and a tier that is **optional on the wire so its absence can be refused**.

Property	Type	Required
`value`	`['null', 'string']`	yes
`visibility`	`ProfileFieldVisibility` or null	yes

`ProfileFieldResponseOfstring`

One field's value and the tier that governs it.

Property	Type	Required
`value`	`['null', 'string']`	yes
`visibility`	`ProfileFieldVisibility`	yes

`ProfileFieldVisibility`

Who may see one profile field — subject V.1's three tiers, SDD §6.2.

Type: `integer`

`QueueEntry`

One row of the vote-ordered queue.

Property	Type	Required
`suggestionId`	`string` (uuid)	yes
`youtubeVideoId`	`string`	yes
`suggestedByUserId`	`string` (uuid)	yes
`votes`	`['integer', 'string']` (int32)	yes
`callerVoted`	`boolean`	yes

`RegistrationRequest`

The registration payload — `docs/api/ENDPOINTS.md` §1.

Property	Type	Required
`email`	`string`	yes
`password`	`string`	yes
`displayName`	`string`	yes

`SessionRequest`

The sign-in payload — `docs/api/ENDPOINTS.md` §1.

Property	Type	Required
`email`	`string`	yes
`password`	`string`	yes

`SessionResponse`

One session, as a client reads it — `docs/api/ENDPOINTS.md` §1.

Property	Type	Required
`sessionId`	`string` (uuid)	yes
`platform`	`string`	yes
`device`	`string`	yes
`appVersion`	`string`	yes
`startedAt`	`string` (date-time)	yes
`expiresAt`	`string` (date-time)	yes

`SocialLinkRequest`

The link payload, which is the sign-in payload — SPEC-041.

Property	Type	Required
`provider`	`string`	yes
`code`	`string`	yes
`codeVerifier`	`string`	yes
`redirectUri`	`string`	yes

`SocialLinkResponse`

One linked provider, as its owner sees it.

Property	Type	Required
`provider`	`string`	yes
`email`	`['null', 'string']`	yes
`linkedAtUtc`	`string` (date-time)	yes

`SocialSessionRequest`

The social sign-in payload — `docs/api/ENDPOINTS.md` §1.

Property	Type	Required
`provider`	`string`	yes
`code`	`string`	yes
`codeVerifier`	`string`	yes
`redirectUri`	`string`	yes

`SuggestionRequest`

The suggestion payload — `docs/api/ENDPOINTS.md` §4.

Property	Type	Required
`youtubeVideoId`	`string`	yes

`TokenGrantResponse`

The successful token response — RFC 6749 §5.1, with OpenID Connect's `id_token`.

Property	Type	Required
`access_token`	`string`	yes
`token_type`	`string`	yes
`expires_in`	`['integer', 'string']` (int32)	yes
`refresh_token`	`string`	yes
`id_token`	`['null', 'string']`	yes
`scope`	`['null', 'string']`	yes

`TokenPairResponse`

The token pair both session endpoints answer with — `docs/api/ENDPOINTS.md` §1.

Property	Type	Required
`accessToken`	`string`	yes
`refreshToken`	`string`	yes
`expiresIn`	`['integer', 'string']` (int32)	yes

`TokenRotationRequest`

The rotation payload — `docs/api/ENDPOINTS.md` §1.

Property	Type	Required
`refreshToken`	`string`	yes

`TrackAddition`

The add-a-track payload — `docs/api/ENDPOINTS.md` §5.

Property	Type	Required
`youtubeVideoId`	`string`	yes
`afterTrackId`	`['null', 'string']` (uuid)	yes
`atHead`	`['null', 'boolean']`	yes

`TrackDestination`

The move payload. Exactly one neighbour, or `422`.

Property	Type	Required
`afterTrackId`	`['null', 'string']` (uuid)	yes
`beforeTrackId`	`['null', 'string']` (uuid)	yes

`TrackMetadata`

One track, as every layer above `Infrastructure` sees it — `docs/api/ENDPOINTS.md` §3.

Property	Type	Required
`youtubeVideoId`	`string`	yes
`title`	`string`	yes
`channel`	`string`	yes
`durationSeconds`	`['integer', 'string']` (int32)	yes
`thumbnailUrl`	`string`	yes

`UpdateProfileRequest`

The profile update payload — `docs/api/ENDPOINTS.md` §2.

Property	Type	Required
`realName`	`ProfileFieldRequestOfstring` or null	yes
`bio`	`ProfileFieldRequestOfstring` or null	yes
`location`	`ProfileFieldRequestOfstring` or null	yes
`birthDate`	`ProfileDateFieldRequest` or null	yes

`UserInfoResponse`

The UserInfo response — OpenID Connect Core 1.0 §5.1, with the members this provider can honour.

Property	Type	Required
sub	string	yes
email	['null', 'string']	yes
email_verified	['null', 'boolean']	yes
name	['null', 'string']	yes

UserSearchResult

One account in a search result — GET /api/v1/users.

Property	Type	Required
userId	string (uuid)	yes
displayName	string	yes

VerificationRequest

The verification resend payload — docs/api/ENDPOINTS.md §1.

Property	Type	Required
email	string	yes

Visibility

Type: integer

VisibleProfile

One account's profile as a **particular reader** may see it — GET /api/v1/users/{userId}.

Property	Type	Required
userId	string (uuid)	yes
displayName	string	yes
realName	['null', 'string']	yes
bio	['null', 'string']	yes
location	['null', 'string']	yes
birthDate	['null', 'string'] (date)	yes

VoteRequest

The vote payload. Empty unless the licence requires presence.

Property	Type	Required
presence	PresenceRequest or null	yes